■ AURACAST — PROJECT REVIEW & VIVA GUIDE

Subject: Multimedia Systems • **Team Size:** 3 Students • **Execution:** Google Chrome / MS Edge

1. HOW AURACAST WORKS (TECHNICAL SUMMARY)

AuraCast transforms a standard webcam into a touchless virtual multimedia studio by coupling **Real-Time Computer Vision** with the **Web Audio API** and a **3-Layer HTML5 Canvas Stack**. No special hardware, depth sensors, or server backends are required.

Pipeline Stage	Core Technology & Mathematics	Syllabus Mapping
1. Vision Sensing	MediaPipe captures 21 3D hand coordinates (X, Y, Z) per frame at 60 FPS from raw video stream.	Module 3 & 4 (Image / Video)
2. Gesture Engine	Calculates Euclidean distances between fingertips and MCP joints to classify 5 static poses (Point, Fist, Palm, Peace, Pinch) + horizontal velocity vectors for dynamic Swipes.	Module 3 (Spatial Geometry)
3. Canvas Stack	3 hardware-accelerated canvas layers: Layer 1 (Video + Sobel convolution filter), Layer 2 (Air Canvas with Bézier smoothing), Layer 3 (Interactive HUD & 3D instrument reticles).	Module 3 & 4 (Graphics & Animation)
4. Audio Synthesis	Client-side polyphonic AudioContext generating synthesized 808 kicks, snare noise bursts, and 8-key melodic synth with dynamic ADSR envelopes + live 128-bin FFT spectrum.	Module 2 (Sound & Audio DSP)
5. QoS Telemetry	Live latency budget monitor (FPS, processing time, hand tracking confidence) with automated fallback to mouse simulation mode.	Module 6 & 7 (Real-Time Systems)

2. WHAT TO EXPLAIN TO MA'AM IN REVIEW (EXACT SCRIPT)

■■ THE 30-SECOND OPENING PITCH (Memorize & Say This First)

*"Good morning Ma'am. Our project is **AuraCast** — a touchless virtual multimedia studio running directly in the web browser. Using just a standard laptop webcam, AuraCast captures 21 3D hand landmarks at 60 FPS, classifies natural spatial gestures, and connects them directly to real-time Web Audio API sound synthesis and AR computer graphics.*

*We designed AuraCast specifically around our **Multimedia Systems syllabus**: it implements client-side **Audio DSP & ADSR envelopes (Module 2)**, **spatial 3x3 Sobel convolution filtering (Module 3)**, **Bézier animation & particle physics (Module 4)**, and **real-time QoS latency monitoring (Modules 6 & 7)**."*

■■ 3-MINUTE ACTION-BY-ACTION DEMO SEQUENCE FOR MA'AM

Time	Action on Screen	What to Say to Ma'am (The Explanation)
0:00 - 0:45	**Air Canvas** Point index finger to draw neon curves in air. Show open palm to erase.	"Here we capture the index tip (landmark 8). We use *Quadratic Bézier interpolation* between frame coordinates to eliminate human tremor, combined with particle physics for glowing trails."
0:45 - 1:30	**Air Synth & Drums** Tap floating drum pads (Kick, Snare, Hi-Hat) & synth keys in mid-air.	"This is Module 2 (Audio). Each pad uses real-time *Web Audio API oscillators and noise buffers with custom ADSR envelopes*. Notice the 128-bin FFT spectrum visualizer at the bottom showing real-time frequency response."
1:30 - 2:15	**Gesture DJ Deck** Closed fist to play/pause. Move hand up/down to change volume. Swipe left/right.	"Here we map continuous spatial coordinates to audio gain (Y-axis = volume). Swiping detects horizontal velocity vectors ($\Delta X/\Delta t$) across a 5-frame sliding window to trigger track skip."
2:15 - 3:00	**Vision Filters & Telemetry** Switch to Sobel Edge filter. Show live telemetry HUD.	"Under Module 3, this executes a real-time *3x3 Sobel spatial convolution kernel* directly on pixel luminance. The bottom HUD proves our QoS: stable 60 FPS with <6ms pipeline latency."

3. TEAM WORK DIVISION (3 STUDENTS) & SYLLABUS MAPPING

When Ma'am asks *"Who did what?"*, present this division clearly. Each student owns a distinct syllabus module:

Student & Role	Syllabus Module	Key Deliverables & Codebase Contribution
Student 1 **Audio & DSP Synthesis Engineer**	**Module 2:** Sound and Audio	• Built the Web Audio API synthesis engine (js/audio-engine.js). • Synthesized 808 Kick, Snare white noise burst, and 8-key melodic synth. • Programmed the 128-bin Fast Fourier Transform (FFT) frequency spectrum visualizer. • Implemented mathematical ADSR (Attack, Decay, Sustain, Release) gain envelopes.
Student 2 **Computer Vision & Graphics Engineer**	**Module 3 & 4:** Images, Graphics & Animation	• Integrated MediaPipe 21-point hand tracking and gesture classifier (js/vision-tracker.js). • Developed Air Canvas with Quadratic Bézier smoothing and particle sparks (js/air-canvas.js). • Programmed pixel-level 3x3 Sobel operator for spatial edge detection (js/filters.js). • Created Cyberpunk and Thermal false-color look-up tables (LUT).
Student 3 **Interactive Systems & Telemetry Lead**	**Module 4, 6 & 7:** Real-Time Systems & QoS	• Engineered 3D physical collision detection for air drum pads and keys (js/instruments.js). • Built the Gesture DJ media deck with elevation volume and velocity swipe skip (js/media-deck.js). • Implemented real-time QoS telemetry HUD (FPS, latency, hand confidence) (js/app.js). • Built hardware-independent Mouse Mode simulation fallback.

4. TOP REVIEW / VIVA QUESTIONS & WINNING ANSWERS

Be prepared for these exact questions from the evaluator during the viva:

Q1: Why did you synthesize audio via Web Audio API instead of just playing pre-recorded MP3 files?

Answer: Pre-recorded MP3 files suffer from decode latency (50-100ms jitter), which destroys real-time musical responsiveness. Procedural synthesis with Web Audio API runs directly on the audio thread with near-zero latency (<5ms) and allows dynamic parameter manipulation (pitch bend, custom ADSR envelopes, filter cutoff modulation) in real time.

Q2: How does the 3x3 Sobel Edge Detection filter work mathematically?

Answer: The Sobel operator computes spatial gradient vectors on image luminance using two 3x3 convolution kernels: Gx = [[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]] for horizontal changes and Gy = [[-1, -2, -1], [0, 0, 0], [1, 2, 1]] for vertical changes. The combined edge magnitude is calculated per pixel as G = sqrt(Gx^2 + Gy^2). If G exceeds a threshold, the pixel is marked as an edge.

Q3: How do you classify a 'Swipe' gesture from a stationary hand movement?

Answer: We track the horizontal velocity vector (Delta_X / Delta_t) of the wrist landmark across a sliding temporal window of 5 frames. A swipe event is dispatched only if horizontal velocity exceeds 0.35 normalized screen width per second and vertical displacement remains bounded under 0.15, avoiding false triggers.

Q4: How does your system eliminate hand tremors while drawing in the air?

Answer: We store recent landmark points and render Quadratic Bézier curves between the midpoints of consecutive coordinate pairs. This smooths out high-frequency biological hand tremors while maintaining low-latency path tracking at 60 FPS.

Q5: How does your project demonstrate Real-Time Systems and QoS (Modules 6 & 7)?

Answer: We enforce a strict 16.6ms frame budget (60 FPS). Our telemetry system monitors camera capture latency, neural inference time, and canvas composite duration. If tracking confidence drops or the camera is unavailable, an automatic QoS fallback activates Mouse Simulation mode to guarantee uninterrupted user experience.

Q6: What coordinate system does MediaPipe hand tracking output?

Answer: MediaPipe returns 21 3D landmarks per hand normalized between 0.0 and 1.0 relative to image width and height. The Z-coordinate represents relative depth from the camera (smaller/negative Z = closer to camera). We map normalized (X, Y) to canvas viewport pixels and use Z-depth for 3D strike velocity.

■ QUICK PRESENTATION CHECKLIST BEFORE REVIEW WITH MA'AM

1. **Audio Activation:** Click the **"■ Enable Sound"** button at top before starting (browsers block audio autoplay until user interaction).
2. **Lighting & Camera:** Ensure decent room lighting facing your hand, holding your hand approx. 0.5 - 1 meter from the webcam.
3. **Emergency Fallback:** If the webcam lags or disconnects during the review, immediately click **"■■ Mouse Mode"** in the top right to continue the demo seamlessly using mouse movement!